

Projet d'Innovation
Année universitaire 2025 – 2026

GoReview AI

Outil CLI d'analyse architecturale de code source

Open source · Local · Multi-langage

Membres du groupe

RAHIOUI Youssef
NAZIH Youssef
Machnaoui Abdellatif
Hattabi Youness
HOUSSY Badr

Mars 2026

1. Présentation du projet

GoReview AI est un outil en ligne de commande (CLI) open source qui analyse un projet informatique et identifie les problèmes architecturaux susceptibles de rendre le code difficile à maintenir sur le long terme.

Contrairement aux linters classiques qui vérifient le style et la syntaxe, GoReview AI raisonne au niveau de l'architecture : il détecte des « mauvaises odeurs » de conception (code smells) et recommande des patrons de conception (design patterns) pour y remédier.

Commande de base

```
$ gorview ./...

[CRITIQUE]  god_struct      service/user.go:14
            UserService · 19 champs · 28 méthodes
            → Patron suggéré : Façade · Effort : Élevé

[MOYEN]     concrete_dep   handler/order.go:22
            OrderHandler.db est un type concret, pas une interface
            → Patron suggéré : Injection de dépendances · Effort : Faible

score de maintenabilité : 61 / 100
```

En résumé

GoReview AI dit non pas « ce code est mal écrit » mais « cette structure va devenir un problème, voici comment l'améliorer et combien d'effort cela demande ».

2. Problème identifié

Les équipes de développement manquent d'un outil rapide, local et gratuit pour évaluer la qualité architecturale d'un projet avant que les problèmes ne deviennent critiques. Les solutions existantes présentent chacune des limites importantes :

Outil	Portée	Architecture	Privé / Local
golangci-lint	Ligne / fonction	Non	Oui
CodeRabbit	Diff de PR	Partielle	Non — cloud
SonarQube	Fichier / classe	Partielle	Lourd à déployer
GoReview AI	Codebase entier	Oui	Oui — 100 % local

Opportunité

Il n'existe pas aujourd'hui d'outil CLI rapide, open source et local qui détecte les problèmes architecturaux et propose des patrons de conception nommés avec une estimation d'effort.

3. Fonctionnement

L'outil fonctionne en deux étapes indépendantes. La première est gratuite et ne nécessite aucune connexion. La seconde est optionnelle et enrichit les résultats grâce à un modèle de langage.

Étape 1 — Analyse statique (sans LLM)

GoReview AI parcourt le code source, construit un arbre syntaxique abstrait (AST) et exécute des détecteurs spécialisés en parallèle. Chaque détecteur repère un type de problème précis :

Problème détecté	Description	Patron suggéré
Classe dieu	Un seul composant gère trop de responsabilités	Façade
Dépendance concrète	Couplage fort à une implémentation spécifique	Injection de dépendances
Complexité élevée	Fonction avec trop de chemins logiques (CC > 10)	Stratégie
Obsession des primitifs	Concepts métier typés comme string ou int	Objet valeur
Chirurgie à la chevrotine	Un changement touche plus de 5 fichiers	Observateur

Étape 2 — Enrichissement LLM (optionnel)

Avec le flag `--llm`, les résultats de l'étape 1 sont envoyés à un modèle de langage (Ollama en local, ou OpenAI/Anthropic en cloud). Le modèle confirme le diagnostic, génère un exemple de refactorisation avant/après, et estime le niveau d'effort.

Respect de la confidentialité

En mode `--llm`, le code source brut n'est jamais transmis au modèle. Seuls les résumés des problèmes détectés sont envoyés (nom du composant, métriques, localisation).

4. Support multi-langage

GoReview AI n'est pas limité au langage Go. Les problèmes architecturaux qu'il détecte sont universels. Seule l'implémentation du détecteur change selon le langage.

Langage	Technologie d'analyse	Version prévue
Go	go/ast + go/types (stdlib)	v1 — natif
TypeScript / JS	TypeScript Compiler API	v2 — planifié
Python	Module ast + libcst	v2 — planifié
Tout langage	tree-sitter (100+ langages)	v2 — fallback universel

tree-sitter est la clé de la couverture large : cette bibliothèque fournit un arbre syntaxique pour plus de 100 langages (Ruby, C++, Rust, Swift, PHP...) sans qu'il soit nécessaire d'écrire un parseur. Cela permet de détecter les problèmes structurels même sans résolution de types complète.

5. Architecture technique

L'architecture est conçue autour d'une séparation claire entre les définitions universelles des problèmes et les détecteurs spécifiques à chaque langage. Ajouter un nouveau langage ou un nouveau type de détection ne nécessite de modifier qu'un seul composant.

```
gorview/  
├─ core/           Définitions universelles (Finding, Score, Report)  
├─ detectors/      Un détecteur par type de problème  
├─ languages/  
│   ├── golang/    Analyse native avec go/ast + go/types  
│   └─ treesitter/ Fallback pour 100+ langages  
└─ llm/           Backends LLM (Ollama, OpenAI)
```

Interface unique

Chaque détecteur implémente une seule interface : Detect(chemin) → []Résultat. Le scoring, l'enrichissement LLM et la génération du rapport héritent automatiquement de tout nouveau détecteur.

Stack technologique

Composant	Technologie
Langage principal	Go
Interface CLI	cobra
Analyse AST (Go)	go/parser, go/types, go/ast (stdlib)
Analyse multi-langage	tree-sitter via go-tree-sitter
LLM local	Ollama (llama3, mistral...)
LLM cloud	OpenAI / Anthropic (optionnel)
Sortie	Terminal coloré, JSON, HTML

6. Plan de développement

Le projet est conçu pour être livré de manière incrémentale. Chaque version apporte une valeur utilisable indépendamment.

Version	Délai estimé	Contenu
v0.1	2–3 semaines	Détecteurs statiques Go (classe dieu, dépendance concrète, complexité). Sortie terminal.
v0.2	5–6 semaines	Backend LLM (Ollama + OpenAI). Conseils de patrons + exemples de refactorisation. Sortie JSON et HTML.
v0.3	2–3 mois	Support TypeScript. Fallback tree-sitter (100+ langages). Mode CI avec seuil de score configurable.
v1.0	3–4 mois	Chirurgie à la chevroline (analyse git). Extension VS Code. API stable pour détecteurs personnalisés.

La version v0.1 constitue un objectif réaliste pour un premier prototype fonctionnel. Go fournit en standard toute l'infrastructure d'analyse syntaxique nécessaire — le travail porte sur la logique de détection et l'interface CLI, pas sur la construction d'un parseur.

7. Valeur ajoutée et originalité

- Local et privé : le code source ne quitte jamais la machine de l'utilisateur, même en mode LLM.
- Gratuit et open source (licence MIT) : aucune infrastructure requise, distribution via un binaire unique.
- Multi-langage dès la v2 : une seule interface pour Go, TypeScript, Python et tout langage supporté par tree-sitter.
- Recommandations actionnables : chaque problème est associé à un patron de conception nommé avec une estimation d'effort (Faible / Moyen / Élevé).
- Intégration CI/CD : le flag `--ci` permet d'échouer automatiquement si le score de maintenabilité est en dessous d'un seuil.

Positionnement

GoReview AI se situe entre le linter (trop bas niveau) et la revue de code humaine (trop coûteuse). Il apporte une valeur architecturale automatisée qui n'existe pas sous cette forme dans l'écosystème open source actuel.

8. Équipe projet

Nom	Filliere
RAHIOUI Youssef	GLSID-2
NAZIH Youssef	GLSID-2

Nom	Filliere
Machnaoui Abdellatif	II-CCN-2
Hattabi Youness	GLSID-2
HOUSSY Badr	II-BDCC-2

GoReview AI · Projet d'Innovation · Mars 2026